

Smart Lender: Loan Approval Prediction System

Project Description:

Smart Lender harnesses the power of Machine Learning to provide intelligent loan approval prediction, enabling banks and financial institutions to make quick, accurate, and data-driven lending decisions. The platform includes a Loan Eligibility Prediction module that analyse applicant information such as gender, marital status, education, income, loan amount, credit history, and property area to determine whether a loan application is likely to be approved or rejected. Utilizing Machine Learning algorithms including Decision Tree, Random Forest, KNN Model, XGBoost Model Smart Lender processes applicant data and generates reliable predictions with high accuracy. Built using Python, Flask, HTML, and CSS, the system provides a secure, responsive, and user-friendly web interface for both applicants and banking professionals.

With proper data preprocessing, feature engineering, and model evaluation techniques, Smart Lender simplifies the loan approval process, reduces manual effort, minimizes risks associated with lending decisions, and helps financial institutions improve operational efficiency through intelligent automation.

Scenarios

Scenario 1: Fast-Track Approval for Low-Risk Applicants

A bank loan officer enters the details of an applicant with a stable income, positive credit history, and strong financial profile into the Smart Lender application. The trained Random Forest model predicts that the loan is likely to be approved. Based on this prediction, the officer can proceed with the application more efficiently, reducing manual evaluation time while maintaining consistency in the preliminary assessment process.

Scenario 2: High-Risk Applicant Detection

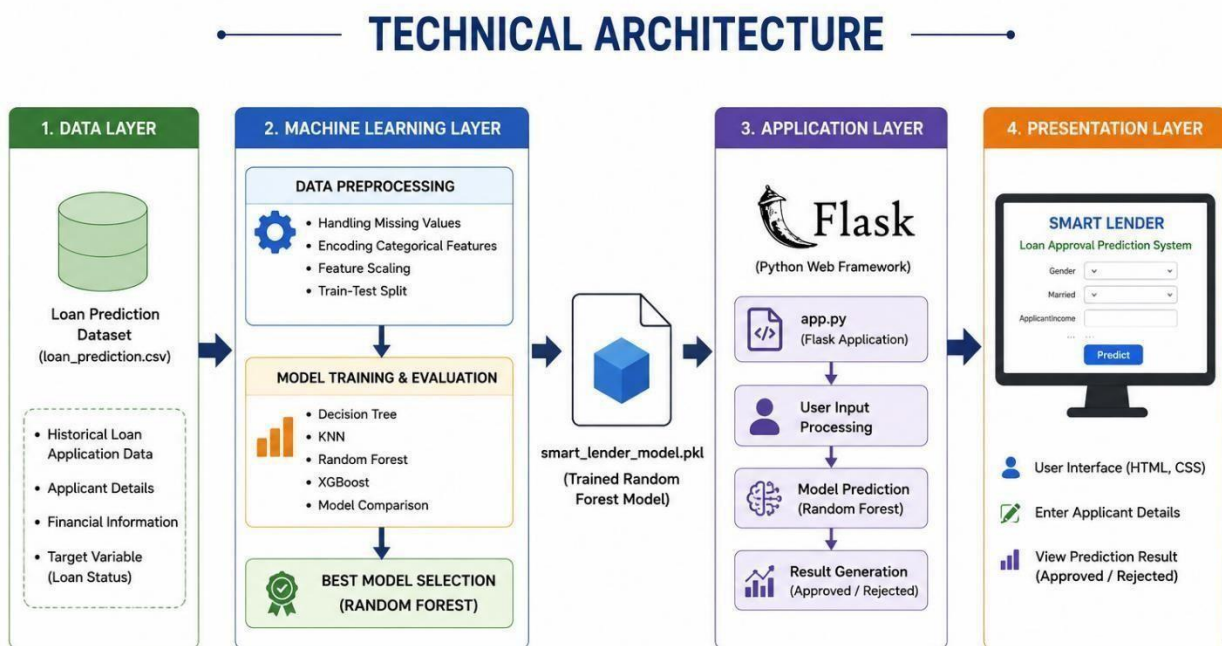
An applicant with irregular self-employment income, a poor credit history, and a high loan amount submits a loan application. The Smart Lender system evaluates the applicant's details and predicts

Loan Rejected due to the high risk of default. The bank can then request additional documents or conduct further verification before making a final decision.

Scenario 3: Bulk Loan Evaluation

During a festive season, a financial institution receives a large number of loan applications. A financial analyst uses the Smart Lender platform to evaluate multiple applicants efficiently. By relying on the XGBoost model's predictions, the system quickly classifies applications as approved or rejected, significantly reducing processing time while maintaining prediction accuracy and supporting informed lending decisions.

Technical Architecture:



Description:

The Smart Lender – Loan Approval Prediction System follows a layered architecture that integrates machine learning with a Flask-based web application to provide accurate loan eligibility predictions. The system begins with the Data Layer, where the historical loan prediction dataset is stored and used for model development. The dataset contains applicant information such as income, credit history, education, loan amount, and other financial attributes required for prediction.

The Machine Learning Layer is responsible for preprocessing the dataset by handling missing values, encoding categorical variables, and splitting the data into training and testing sets. Multiple machine learning algorithms, including Decision Tree, K-Nearest Neighbors (KNN), Random Forest, and XGBoost, are trained and evaluated. After comparing their performance, the Random Forest algorithm is selected as the final prediction model and saved as a Pickle (.pkl) file.

The Application Layer consists of the Flask web application, which loads the trained model and processes user input. When an applicant enters the required details, the application validates the information, sends it to the trained model for prediction, and generates the loan approval result. The Presentation Layer provides a simple and user-friendly interface developed using HTML and CSS.

Users can enter applicant details through the web form and instantly receive the predicted loan approval status. The completed application is version-controlled using GitHub.

Pre-requisites:

Hardware Requirements

- Processor: Intel Core i3 or above
- RAM: 4 GB minimum
- Storage: 10 GB free space

Software Requirements

- Flask Framework Knowledge: <https://flask.palletsprojects.com/en/stable/>
- Scikit-learn Documentation: <https://scikit-learn.org/stable/>
- Pandas Documentation: <https://pandas.pydata.org/docs/>
- NumPy Documentation: <https://numpy.org/doc/>
- XGBoost Documentation: <https://xgboost.readthedocs.io/en/stable/>
- Matplotlib Documentation: <https://matplotlib.org/stable/contents.html> (matplotlib.org in Bing)
- Seaborn Documentation: <https://seaborn.pydata.org/>
- SciPy Documentation: <https://docs.scipy.org/doc/scipy/>
- Python Official Documentation: <https://docs.python.org/3/>
- Git Documentation: <https://git-scm.com/doc>

Project Workflow

Milestone 1: Dataset Preparation and System Architecture

- Activity 1.1: Obtain and analyze the Loan Prediction dataset and understand its features.
 - Activity 1.2: Research and select suitable Machine Learning algorithms for loan prediction.
 - Activity 1.3: Define the application architecture, including frontend, backend, and machine learning integration.
 - Activity 1.4: Set up the development environment and install the required libraries and dependencies.
-

Milestone 2: Data Preprocessing and Model Development

- Activity 2.1: Perform data preprocessing, including handling missing values, encoding categorical data, and feature scaling.
 - Activity 2.2: Train and evaluate multiple machine learning models such as Decision Tree, Random Forest, KNN, and XGBoost.
-

Milestone 3: Model Integration and App.py Development

- Activity 3.1: Develop the main application logic in app.py, load the trained model, establish routes, and implement prediction functionality.
-

Milestone 4: Frontend Development

- Activity 4.1: Design and develop a responsive user interface using HTML, and CSS.
- Activity 4.2: Create dynamic templates using Flask's render_template to display prediction results.

Milestone 5: Deployment (Optional(NA))

- Activity 5.1: Prepare the application for local deployment and verify all dependencies and configurations.
 - Activity 5.2: Deploy the application on IBM Cloud to provide secure and scalable access.
-

Milestone 6: Conclusion

- Evaluate the performance of the Smart Lender system and demonstrate how machine learning can improve loan approval decisions by providing fast, accurate, and data-driven predictions.

Milestone 1: Model Selection and Architecture

In this milestone, the Smart Lender – AI Powered Loan Approval Prediction System focuses on understanding the Skill Wallet Loan Prediction dataset, selecting the best machine learning model, and designing the application architecture. The dataset is analyzed to identify important features affecting loan approval decisions.

Different machine learning algorithms such as Decision Tree, Random Forest, K-Nearest Neighbors (KNN), and XGBoost are evaluated based on their prediction performance. After comparison, XGBoost is selected as the final model because it provides the highest accuracy for loan approval prediction.

The system architecture consists of a Frontend (HTML/CSS) for user interaction, a Flask Backend for processing requests, and a Machine Learning Layer containing the trained XGBoost model for real-time predictions.

Activity 1.1: Dataset Collection and Analysis

The Loan Prediction dataset provided by Skill Wallet is used for developing the Smart Lender system. The dataset contains information about applicants, including demographic, financial, and credit-related attributes that influence loan approval decisions.

Activities Performed:

- Download and import the Skill Wallet loan dataset.
- Study the dataset structure and identify input and output variables.
- Analyze features such as Gender, Marital Status, Education, Applicant Income, Loan Amount, Credit History, and Property Area.
- Identify and handle missing values using suitable preprocessing techniques.
- Understand the distribution of loan approval and rejection cases.

Step 1 – Obtain the Dataset

Download the Loan Prediction Dataset provided by Skill Wallet and place the loan_prediction.csv file inside the Dataset folder.

Step 2 – Load the Dataset

Import the dataset using the Pandas library.

Step 3 – Explore the Dataset

Check the number of rows, columns, and data types.

Step 4 – Identify Missing Values

Analyze the dataset for missing values

Step 5 – Understand the Features

Study important attributes such as:

- Gender
- Marital Status
- Applicant Income
- Loan Amount
- Credit History
- Property Area
- Loan Status

Step 6 - Prepare the Dataset

Perform preprocessing by handling missing values and preparing the data for model training.

Activity 1.2: Research and Selection of Machine Learning Model

Since loan approval prediction is a classification problem, multiple machine learning algorithms are studied and compared to identify the best-performing model.

Models Evaluated:

- Decision Tree Classifier
 - Random Forest Classifier
 - K-Nearest Neighbors (KNN)
 - XGBoost Classifier
- Activities Performed.
- Train each model using the preprocessed dataset.
 - Compare models based on training and testing accuracy.
 - Evaluate the models' ability to predict loan approval correctly.
 - Analyze model performance and select the most suitable algorithm.

Final Model Selected: XGBoost Classifier

Reason for Selection:

XGBoost achieved the highest prediction accuracy (94.7% training accuracy and 81.1% testing accuracy) and provided better performance compared to the other models.

Activity 1.3: Define the Architecture of the Application

The Smart Lender application follows a three-layer architecture to provide real-time loan approval predictions.

Frontend Layer

- Developed using HTML and CSS.
- Provides forms for entering applicant information.
- Displays loan approval or rejection results.

Backend Layer

- Developed using Python Flask.
- Receives user inputs and processes the data.
- Loads the trained machine learning model and scaler.

Machine Learning Layer

- Contains the trained XGBoost model and preprocessing components.
- Predicts whether the loan application is approved or rejected.

System Workflow

1. User enters loan details.
2. Flask receives and preprocesses the input.
3. The trained XGBoost model predicts the result.
4. The prediction is displayed on the web page.

Activity 1.4: Set Up the Development Environment

The development environment is configured to build, train, and deploy the Smart Lender application efficiently.

Steps:

- Install Python 3.10 or above and Visual Studio Code.
- Create a virtual environment to manage project dependencies.
- Install required libraries such as Flask, Pandas, NumPy, Scikit-learn, XGBoost, and Joblib.

- Create the project folder structure for datasets, training scripts, templates, and static files.
- Verify that all dependencies are installed successfully and the environment is ready for development.

Objective:

To establish a stable and organized environment for developing and testing the Smart Lender application.

Activity 1.5: Design the Loan Prediction Workflow

A workflow is designed to define how data moves through the Smart Lender system, from user input to prediction output.

Steps:

- Accept applicant details through the web interface.
- Validate and preprocess the input data.
- Apply feature scaling using the saved scaler.
- Pass the processed data to the trained XGBoost model.
- Generate the prediction result as Loan Approved or Loan Rejected.
- Display the prediction result to the user through the web application.

Objective:

To create a structured workflow that ensures accurate, efficient, and real-time loan approval prediction.

Milestone 2: Data Preprocessing and Model Development

Theory

In this milestone, the focus is on preparing the Skill Wallet Loan Prediction Dataset for machine learning and developing different classification models for loan approval prediction. Since the dataset contains missing values and categorical features, preprocessing techniques are applied to improve the quality of the data and increase prediction accuracy.

After preprocessing, multiple machine learning algorithms such as Decision Tree, Random Forest, K-Nearest Neighbors (KNN), and XGBoost are trained and evaluated. The performance of each model is compared to identify the best algorithm for predicting loan eligibility.

This milestone plays an important role in building an accurate and reliable loan approval prediction system.

Activity 2.1: Data Preprocessing

Data preprocessing is performed to convert the raw dataset into a clean and structured format suitable for machine learning.

Steps:

Step 1 – Load the Dataset

Import the dataset using the Pandas library.

```
import pandas as pd
df = pd.read_csv("Dataset/loan_prediction.csv")
```

Step 2 – Handle Missing Values

Identify and fill missing values using suitable techniques.

```
print(df.isnull().sum()) df.fill(inplace=True)
```

Step 3 – Remove Unnecessary Columns

Remove columns that do not contribute to prediction.

```
df.drop("Loan_ID", axis=1, inplace=True)
```

Step 4 – Encode Categorical Variables

Convert categorical features into numerical values.

```
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
```

Step 5 – Split Features and Target Variable

```
X = df.drop("Loan_Status", axis=1) y
= df["Loan_Status"]
```

Step 6 – Feature Scaling

Normalize the input features using StandardScaler.

```
from sklearn.preprocessing import StandardScaler scaler
= StandardScaler()
```

Activity 2.2: Train Machine Learning Models

Different machine learning algorithms such as Decision Tree, Random Forest, K-Nearest Neighbors (KNN), and XGBoost are trained using the preprocessed dataset. The models are evaluated based on their prediction accuracy to identify the best algorithm for loan approval prediction.

Step 1 – Split the Dataset

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Step 2 – Train the Model

```
from xgboost import XGBClassifier
model = XGBClassifier()
model.fit(X_train, y_train)
```

Activity 2.3: Model Performance Evaluation

The performance of each model is compared using training and testing accuracy.

Model	Training Accuracy	Testing Accuracy
Decision Tree	100%	70%
Random Forest	100%	79%

KNN Model	83%	79%
XGBoost	94%	81%

Analysis:

- Decision Tree and Random Forest show signs of overfitting.
- KNN provides moderate performance.
- XGBoost achieves the highest testing accuracy and better generalization.

Objective:

To identify the most suitable model for loan approval prediction.

Activity 2.4: Selection of the Final Model

After evaluating all models, XGBoost Classifier is selected as the final prediction model.

Reasons for Selection:

- Highest testing accuracy.
- Better handling of complex relationships.
- Reduced overfitting.
- Improved prediction performance.

Outcome:

The XGBoost model is finalized for integration into the Smart Lender web application.

Activity 2.5: Save the Trained Model

The final model and scaler are saved for future predictions.

Steps:

Save the Model

```
joblib.dump(model, "model.pkl")
```

Save the Scaler

```
joblib.dump(scaler, "scaler.pkl")
```

Milestone 3: Model Integration and App.py Development

Theory

In this milestone, the trained XGBoost model and scaler are integrated into the Flask application to provide real-time loan approval predictions. The app.py file acts as the main component of the application by handling user requests, processing input data, loading the trained model, and displaying prediction results through web pages.

This milestone ensures smooth communication between the frontend and the machine learning model, enabling users to enter applicant details and instantly receive loan approval predictions.

Activity 3.1: Load the Trained Model

The saved model and scaler are loaded into the Flask application for prediction.

Step 1 – Load Model and Scaler

```
model = joblib.load("model.pkl")  
scaler = joblib.load("scaler.pkl")
```

Activity 3.2: Create Flask Routes

Flask routes are created to display the home page, input form, and prediction result.

Step 2 – Create Home Route

```
@app.route("/") def  
home():  
    return render_template("index.html")
```

Activity 3.3: Implement Prediction Logic

The application receives user inputs, preprocesses the data, and generates the prediction result.

Step 3 – Generate Prediction

```
prediction = model.predict(data)
```

Milestone 4: Frontend Development

Theory

This milestone focuses on developing the user interface of the Smart Lender application. The frontend acts as a bridge between the user and the machine learning model by allowing users to enter applicant details and view the loan prediction results in an easy and interactive manner.

The frontend is developed using HTML, and CSS, to create responsive and visually appealing web pages. Multiple web pages are designed, including the Home Page, Loan Application Form, and Prediction Result Page. These pages provide a simple navigation flow and improve the overall user experience.

The Flask framework is used to connect the frontend with the backend through dynamic templates. Flask's `render_template()` function displays different pages and passes the prediction result generated by the machine learning model to the user interface.

Special attention is given to the design of the application by using modern layouts, colors, images, and responsive components so that. The frontend also validates user inputs and ensures that all required details are provided before making a prediction.

The successful completion of this milestone results in a fully functional and user-friendly web application that enables users to interact with the Smart Lender system and obtain realtime loan approval predictions efficiently.

Activity 4.1: Design and Develop the User Interface

The user interface is designed to provide a simple and responsive experience for users.

Features Developed:

- Index Page
- Result Page

Step 1 – Create Index Page

```
@app.route("/") def  
home():  
    return render_template("index.html")
```

Step 2 – Create Result Page

```
@app.route("/predict")  
def predict():  
    return render_template("result.html")
```

Activity 4.2: Create Dynamic Templates

Dynamic templates are created using Flask to display prediction results based on user input.

Step 1 – Render the Result Page

```
return render_template("result.html", prediction=result)
```

Step 2 – Display Prediction in HTML

```
<h2>{{ prediction }}</h2>
```

Milestone 5: Deployment (Optional NA)

Theory

In this milestone, the Smart Lender – AI Powered Loan Approval Prediction System is prepared for deployment so that users can access the application through a web browser. Deployment involves configuring the Flask application, ensuring all required libraries and dependencies are installed, and integrating the trained machine learning model with the web application.

The application is designed to be deployed on IBM Cloud, which provides a secure and scalable platform for hosting the project. Before deployment, the application is thoroughly tested in the local environment to ensure that all functionalities, including data input, prediction generation, and result display, work correctly.

The deployment process makes the Smart Lender system accessible to banking professionals and financial institutions, allowing them to perform real-time loan approval predictions from anywhere.

Activity 5.1: Prepare the Application for Deployment

The application is configured and tested in the local environment before deployment.

Steps:

- Verify that all project files are properly organized.
- Ensure all required libraries are installed.
- Test the Flask application locally.
- Verify that the trained model and scaler load successfully.

Step 1 – Install Dependencies

```
pip install -r requirements.txt
```

Step 2 – Run the Flask Application

```
python app.py
```

Activity 5.2: Deploy the Application on IBM Cloud

The Smart Lender application is deployed on IBM Cloud to provide public access.

Steps:

- Create an IBM Cloud account.
- Configure the deployment environment.
- Upload the project files to IBM Cloud.
- Deploy the Flask application and machine learning model.
- Test the deployed application using the public URL.

Step 1 – Login to IBM Cloud

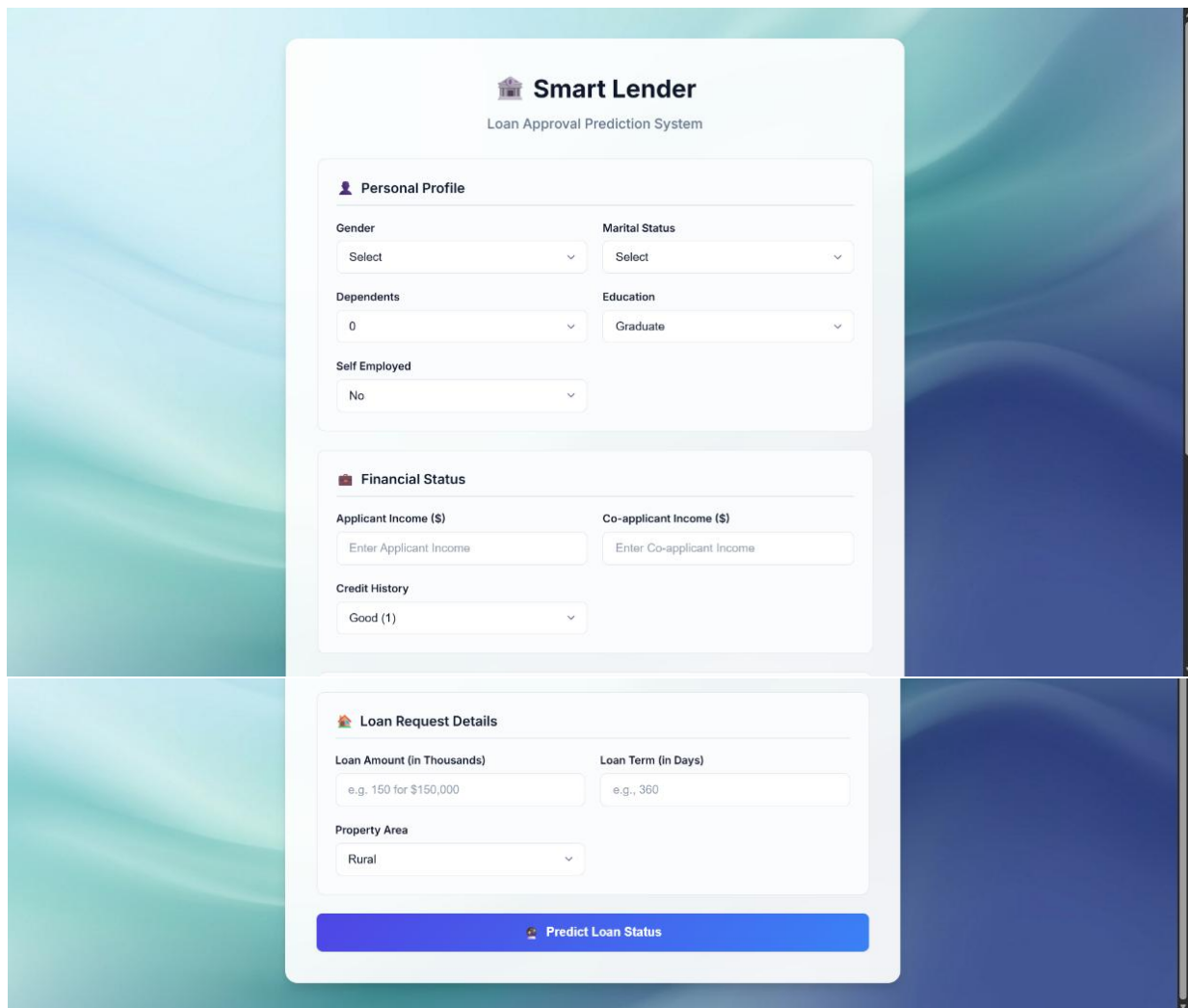
```
ibmcloud login
```

Step 2 – Deploy the Application

```
ibmcloud cf push
```


Exploring the Website's Web Pages:

Index / Input Page:



The image shows a web application interface for a loan approval prediction system. The page has a light blue background with a white central form area. The form is titled "Smart Lender" with a house icon and the subtitle "Loan Approval Prediction System". It is divided into three main sections: "Personal Profile", "Financial Status", and "Loan Request Details".

Personal Profile

- Gender: Select (dropdown menu)
- Marital Status: Select (dropdown menu)
- Dependents: 0 (dropdown menu)
- Education: Graduate (dropdown menu)
- Self Employed: No (dropdown menu)

Financial Status

- Applicant Income (\$): Enter Applicant Income (text input)
- Co-applicant Income (\$): Enter Co-applicant Income (text input)
- Credit History: Good (1) (dropdown menu)

Loan Request Details

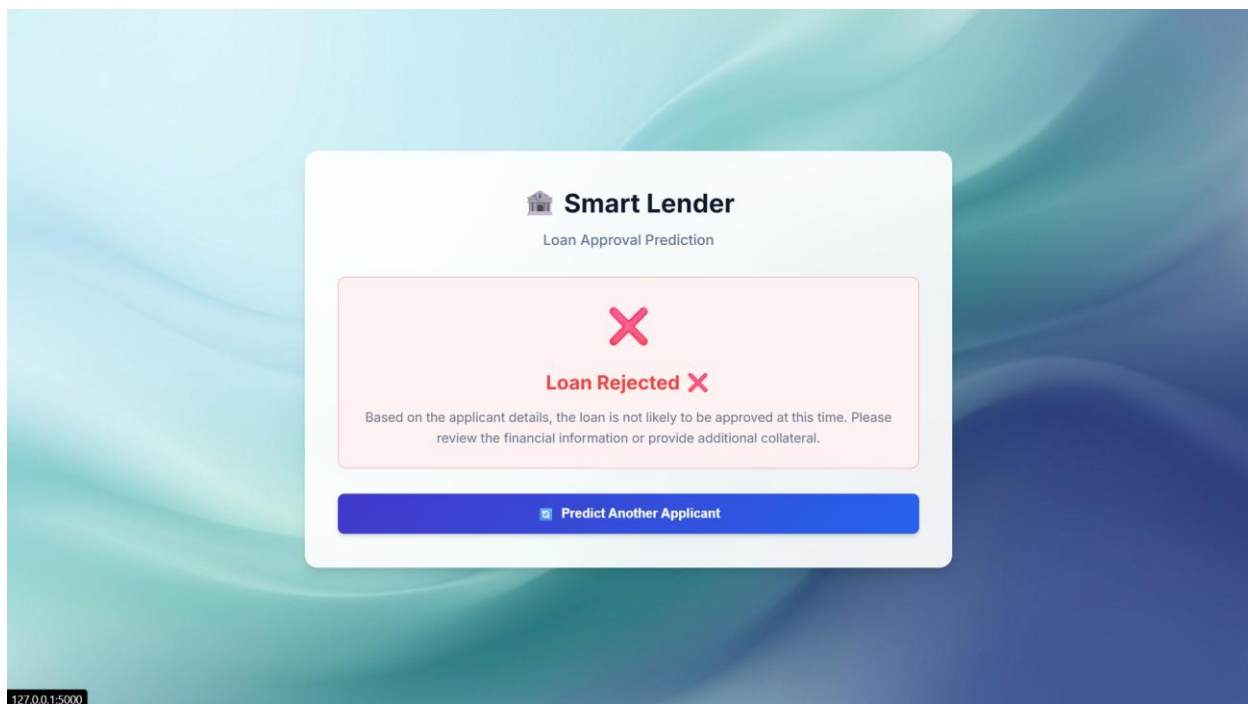
- Loan Amount (in Thousands): e.g., 150 for \$150,000 (text input)
- Loan Term (in Days): e.g., 360 (text input)
- Property Area: Rural (dropdown menu)

At the bottom of the form is a blue button labeled "Predict Loan Status" with a small house icon.

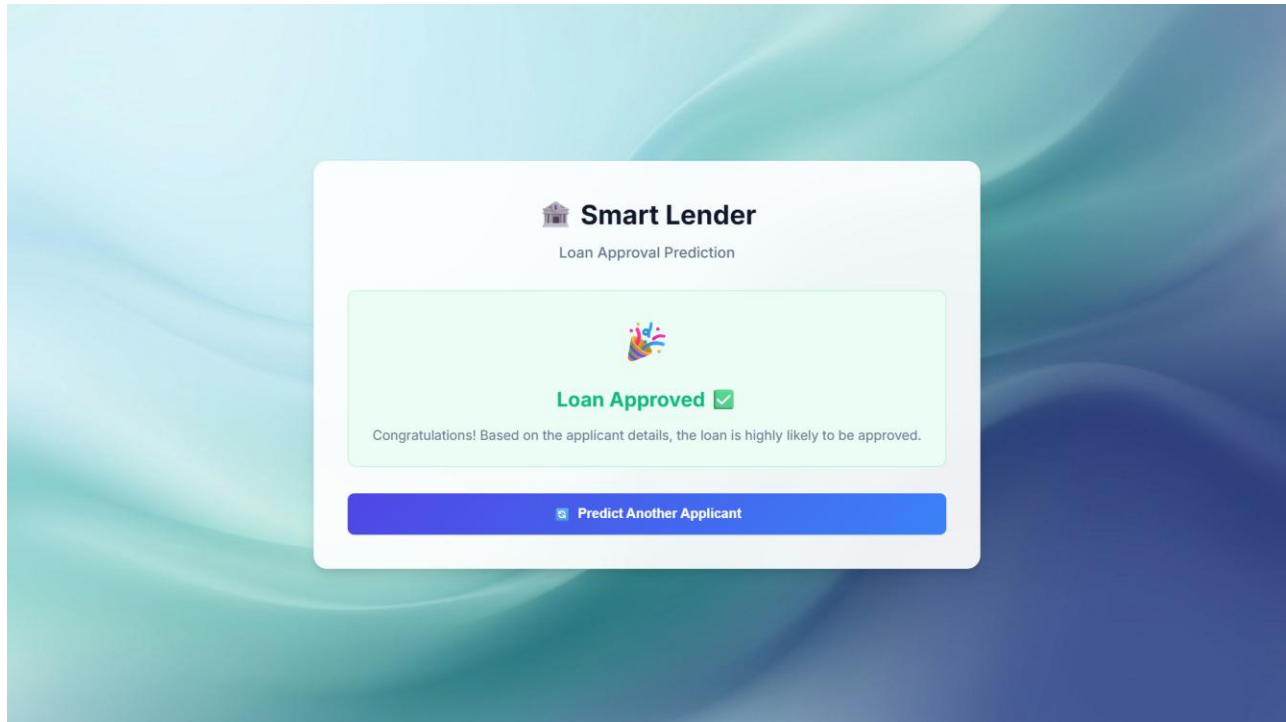
Description:

The Loan Application Form allows users to enter applicant details such as personal information, income, loan amount, loan term, and credit history. After submitting the form, the data is processed by the XGBoost machine learning model, which predicts whether the loan application will be approved or rejected. The page provides a simple, responsive, and user-friendly interface for real-time loan prediction.

Results Section(When Rejected):



Result Section(When Approved):



Description:

The Result Page displays the loan prediction generated by the Smart Lender system. Based on the applicant's details, the trained XGBoost model predicts whether the loan is approved or rejected. In the above result, the applicant's loan is approved. The page also provides a "Predict Another Loan" button to perform a new prediction easily.

Conclusion:

The Smart Lender – AI Powered Loan Approval Prediction System is an intelligent web-based application developed to predict loan approval status using Machine Learning techniques. The system analyzes important applicant information such as gender, marital status, income, education, credit history, loan amount, and property area to determine whether a loan application is likely to be approved or rejected. The project uses the XGBoost algorithm, which provides high prediction accuracy and better performance compared to other machine learning models. The application is

developed using Python, Flask, HTML, CSS, and JavaScript, providing a simple, interactive, and user-friendly interface for users and banking professionals.

By automating the loan eligibility prediction process, the system reduces manual effort, saves time, minimizes human errors, and helps financial institutions make faster and more reliable lending decisions. The project also demonstrates the practical application of Artificial Intelligence and Machine Learning in the banking sector. In the future, the system can be further enhanced by integrating real-time banking databases, customer authentication, document verification, and advanced deep learning techniques to improve prediction accuracy and support smarter financial decision-making.